

STRINGS FOR MNCs & BANKS — DSA PREPARATION NOTES

This comprehensive reference document contains core concepts, advanced algorithms, optimized code implementations (in C++ and Java), and high-yield interview questions designed to help you ace technical rounds at major MNCs (Google, Amazon, Microsoft) and investment banks (Goldman Sachs, Morgan Stanley, JPMorgan Chase).

1. String Fundamentals & Memory Representation

Understanding how strings are stored in memory and how different languages handle string modification is critical for optimization. A single incorrect string operations loop can degrade a $O(N)$ algorithm into a $O(N^2)$ disaster.

C++ vs. Java: Core Differences

Feature	C++ (<code>std::string</code>)	Java (<code>java.lang.String</code>)
Mutability	Mutable (can change in-place)	Immutable (cannot be modified after creation)
Allocation	Typically Heap (uses Small String Optimization for small sizes)	Heap (utilizes the String Constant Pool)
Pass-by-value	Copies the entire string ($O(N)$ time)	Copies the reference copy ($O(1)$ time)
Modification	Modifies in-place: <code>s[0] = 'a'</code> ($O(1)$)	Creates a new String object ($O(N)$)

Memory Optimization: Java String Constant Pool (SCP)

In Java, JVM optimizes memory by storing exactly one copy of each literal string in a special area of the heap called the **String Constant Pool (SCP)**. - `String s1 = "hello";` — Creates "hello" in the SCP (if not present) and points `s1` to it. - `String s2 = "hello";` — Reuses the existing "hello" in the SCP. `s1 == s2` is `true`. - `String s3 = new String("hello");` — Forces the creation of a new String object on the heap. `s1 == s3` is `false`. To move it to pool manually, use `s3.intern()`.

Interview Warning: In Java, because strings are immutable, performing repeated concatenations in a loop like `str += char` creates a new String object at each step, resulting in an overall time complexity of $O(N^2)$. To fix this, always use `StringBuilder` (single-threaded, fast) or `StringBuffer` (thread-safe, synchronized), which performs in-place modifications in $O(1)$ amortized time.

Complexity of Basic Operations

- **Length Lookup:** $O(1)$ in both C++ (`s.length()`) and Java (`s.length()`).
 - **Accessing i -th Character:** $O(1)$ in C++ (`s[i]`) and Java (`s.charAt(i)`).
 - **Concatenation:** $O(N+M)$ where N and M are lengths of the strings.
 - **Substring Extraction:** $O(L)$ where L is the length of the substring. (In C++ `s.substr(pos, len)`; in Java `s.substring(start, end)`).
-

2. Core Pattern Matching & Advanced Algorithms

Pattern matching involves finding occurrences of a pattern string P of length M inside a text string T of length N . A naive matching algorithm takes $O(N \cdot M)$ time. MNCs frequently ask for linear-time pattern matching.

2.1 Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm performs pattern matching in $O(N + M)$ time and $O(M)$ space. It avoids backtracking in the text T by leveraging preprocessed information about the pattern P .

The Longest Prefix Suffix (LPS) Array

The key to KMP is the `lps` array. `lps[i]` stores the length of the longest proper prefix of $P[0..i]$ that is also a suffix of $P[0..i]$.

LPS Array Construction Logic: We maintain two pointers: `len` (tracks the length of the current longest prefix suffix) and `i` (tracks the current character in pattern).
1. If `P[i] == P[len]`, we increment `len` and assign `lps[i] = len`, then increment `i`.
2. If `P[i] != P[len]`: - If `len != 0`, we fallback: `len = lps[len - 1]`. We do not increment `i`. - If `len == 0`, we set `lps[i] = 0` and increment `i`.

KMP Implementation (C++)

```

#include <iostream>
#include <vector>
#include <string>

using namespace std;

// Compute LPS Array
vector<int> computeLPS(const string& pat) {
    int m = pat.length();
    vector<int> lps(m, 0);
    int len = 0;
    int i = 1;
    while (i < m) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

// KMP Pattern Search
vector<int> KMPSearch(const string& txt, const string& pat) {
    int n = txt.length();
    int m = pat.length();
    vector<int> lps = computeLPS(pat);
    vector<int> result;

    int i = 0; // index for txt
    int j = 0; // index for pat
    while (i < n) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }
        if (j == m) {
            result.push_back(i - j); // Match found at index (i - j)
            j = lps[j - 1];
        } else if (i < n && pat[j] != txt[i]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return result;
}

```

2.2 Rabin-Karp Algorithm (Rolling Hash)

The Rabin-Karp algorithm uses **hashing** to find pattern matches. It achieves $O(N + M)$ average-case time complexity and $O(1)$ auxiliary space.

Core Concept: Rolling Hash

We compute the hash of the pattern P and the hash of all substrings of T of length M . If a substring's hash matches the pattern's hash, we do a character-by-character validation to handle potential collisions.

Using a polynomial rolling hash: $H(S) = (S[0] \cdot d^{M-1} + S[1] \cdot d^{M-2} + \dots + S[M-1] \cdot d^0) \pmod{q}$ where d is the alphabet size (typically 256) and q is a large prime (e.g., 10^9+7).

To slide the hash window from index i to $i+1$: $H_{\text{new}} = \left((H_{\text{old}} - T[i] \cdot d^{M-1}) \cdot d + T[i+M] \right) \pmod{q}$

Rabin-Karp Implementation (Java)

```

import java.util.ArrayList;
import java.util.List;

public class RabinKarp {
    private static final int D = 256; // Alphabet size
    private static final int Q = 1000000007; // Large prime

    public static List<Integer> search(String txt, String pat) {
        List<Integer> matches = new ArrayList<>();
        int n = txt.length();
        int m = pat.length();
        if (m > n) return matches;

        long patHash = 0;
        long txtHash = 0;
        long h = 1; // Value of d^(m-1) % q

        // Calculate d^(m-1) % q
        for (int i = 0; i < m - 1; i++) {
            h = (h * D) % Q;
        }

        // Calculate initial hashes
        for (int i = 0; i < m; i++) {
            patHash = (D * patHash + pat.charAt(i)) % Q;
            txtHash = (D * txtHash + txt.charAt(i)) % Q;
        }

        // Slide pattern over text
        for (int i = 0; i <= n - m; i++) {
            if (patHash == txtHash) {
                // Potential match, check characters
                int j;
                for (j = 0; j < m; j++) {
                    if (txt.charAt(i + j) != pat.charAt(j)) {
                        break;
                    }
                }
                if (j == m) {
                    matches.add(i);
                }
            }

            // Compute hash for next window
            if (i < n - m) {
                txtHash = (D * (txtHash - txt.charAt(i) * h) + txt.charAt(i + m)) % Q;
                if (txtHash < 0) {
                    txtHash = txtHash + Q;
                }
            }
        }
        return matches;
    }
}

```

2.3 Z-Algorithm

The Z-Algorithm computes a **Z-array** where **Z[i]** is the length of the longest substring starting from **S[i]** which is also a prefix of **SS**. The algorithm computes this in **O(N)** time and **O(N)** space.

Algorithm Execution

To find pattern P in text T : 1. Create a concatenated string $S = P + \text{"\$"} + T$, where $\$$ is a sentinel character not present in either string. 2. Build the Z-array for S . 3. Any index i where $Z[i] == M$ (length of pattern) indicates a match at index $i - M - 1$ in the original text.

To build the Z-array in linear time, we maintain a search window $[L, R]$ (called the Z-box) which matches a prefix of S .

3. String Dynamic Programming & Sliding Window

These two algorithmic paradigms are highly tested in banking interviews (Goldman Sachs, Morgan Stanley) where algorithmic rigour is prioritized.

3.1 Sliding Window / Two Pointers

The sliding window technique maintains a dynamic subarray over a string to find substrings satisfying certain properties (e.g., uniqueness, character count).

Framework for Minimum Window Substring

```
int l = 0, r = 0;
unordered_map<char, int> map; // Tracks target string frequencies
int required = map.size();
int formed = 0;
while (r < s.length()) {
    char c = s[r];
    // Add s[r] to window, update formed variables
    r++;
    while (formed == required) {
        // Update minimum window length and answer
        char leftChar = s[l];
        // Remove s[l] from window, update formed variables
        l++;
    }
}
```

3.2 String Dynamic Programming

Many string comparisons (e.g., spell checking, genetic sequencing) require dynamic programming.

Longest Common Subsequence (LCS)

Find the length of the longest subsequence present in both strings S_1 and S_2 .

- **State:** $dp[i][j]$ is the length of the LCS of prefixes $S_1[0..i-1]$ and $S_2[0..j-1]$.
- **Recurrence:** $dp[i][j] = \begin{cases} 1 + dp[i-1][j-1] & \text{if } S_1[i-1] == S_2[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{if } S_1[i-1] \neq S_2[j-1] \end{cases}$

Edit Distance (LeetCode 72)

Given two strings S_1 and S_2 , find the minimum number of operations (insert, delete, replace) required to convert S_1 to S_2 .

- **Recurrence:**
$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] & \text{if } S1[i-1] == S2[j-1] \\ 1 + \min(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{if } S1[i-1] \neq S2[j-1] \end{cases}$$
-

4. Trie (Prefix Tree) Data Structure

A Trie is a tree-like data structure used to store retrieval keys over a string alphabet. It is optimized for prefix-based queries (e.g., search autocomplete, dictionary spell checkers).

Why use a Trie?

If we have N strings of maximum length L , searching a string of length L in a Hash Set takes $O(L)$ time on average. However, finding all strings with a prefix of length K in a Hash Set requires scanning all N elements ($O(N \cdot L)$). A Trie solves prefix queries in **$O(K)$ time**.

Trie Implementation (Java)

```

class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord = false;
}

public class Trie {
    private final TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    // Inserts a word into the trie. Time: O(L), Space: O(L)
    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) {
                curr.children[idx] = new TrieNode();
            }
            curr = curr.children[idx];
        }
        curr.isEndOfWord = true;
    }

    // Returns if the word is in the trie. Time: O(L), Space: O(1)
    public boolean search(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) {
                return false;
            }
            curr = curr.children[idx];
        }
        return curr.isEndOfWord;
    }

    // Returns if there is any word in the trie that starts with the given prefix. Time: O(K), Space: O(1)
    public boolean startsWith(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) {
                return false;
            }
            curr = curr.children[idx];
        }
        return true;
    }
}

```

5. Top MNC & Bank Placement Questions

These handpicked questions represent the most frequently asked string problems in technical interview loops.

Question 5.1: Minimum Window Substring (LeetCode 76 — Hard)

Target Companies: Google, Amazon, Goldman Sachs, Uber

Given two strings **s** and **t** of lengths **m** and **n** respectively, return the minimum window substring of **s** such that every character in **t** (including duplicates) is included in the window. If there is no such substring, return the empty string **""**.

C++ Optimized Solution (Sliding Window, $O(M + N)$ Time, $O(1)$ Auxiliary Space)

```
#include <string>
#include <vector>
#include <climits>

using namespace std;

string minWindow(string s, string t) {
    if (s.empty() || t.empty()) return "";

    vector<int> tFreq(128, 0);
    for (char c : t) tFreq[c]++;

    vector<int> windowFreq(128, 0);
    int required = 0;
    for (int count : tFreq) {
        if (count > 0) required++;
    }

    int l = 0, r = 0;
    int formed = 0;
    int minLen = INT_MAX;
    int startIdx = 0;

    while (r < s.length()) {
        char c = s[r];
        windowFreq[c]++;

        if (tFreq[c] > 0 && windowFreq[c] == tFreq[c]) {
            formed++;
        }

        while (l <= r && formed == required) {
            c = s[l];
            if (r - l + 1 < minLen) {
                minLen = r - l + 1;
                startIdx = l;
            }

            windowFreq[c]--;
            if (tFreq[c] > 0 && windowFreq[c] < tFreq[c]) {
                formed--;
            }
            l++;
        }
        r++;
    }

    return minLen == INT_MAX ? "" : s.substr(startIdx, minLen);
}
```

Question 5.2: Longest Palindromic Substring (LeetCode 5 — Medium)

Target Companies: Morgan Stanley, Amazon, Microsoft, JP Morgan

Given a string **s**, return the longest palindromic substring in **s**.

Java Optimized Solution (Expand Around Center, $O(N^2)$ Time, $O(1)$ Space)

```

public class Solution {
    public String longestPalindrome(String s) {
        if (s == null || s.length() < 1) return "";
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            int len1 = expandAroundCenter(s, i, i); // Odd length palindromes (e.g. aba)
            int len2 = expandAroundCenter(s, i, i + 1); // Even length palindromes (e.g. abba)
            int len = Math.max(len1, len2);
            if (len > end - start) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }
        return s.substring(start, end + 1);
    }

    private int expandAroundCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        // Returns the length of the matching palindrome substring
        return right - left - 1;
    }
}

```

Question 5.3: Group Anagrams (LeetCode 49 — Medium)

Target Companies: Goldman Sachs, Amazon, Facebook

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

C++ Optimized Solution ($O(N \cdot L \log L)$ Time, $O(N \cdot L)$ Space where L is max word length)

```

#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>

using namespace std;

vector<vector<string>>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (const string& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        groups[key].push_back(s);
    }

    vector<vector<string>>> result;
    result.reserve(groups.size());
    for (auto& pair : groups) {
        result.push_back(move(pair.second));
    }
    return result;
}

```

Question 5.4: Decode String (LeetCode 394 — Medium)

Target Companies: Google, Amazon, JP Morgan

Given an encoded string, return its decoded string. The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times.

Java Solution (Stack-based Parsing, $O(N)$ Time and Space where N is decoded length)

```
import java.util.Stack;

public class DecodeStringSolution {
    public String decodeString(String s) {
        Stack<Integer> countStack = new Stack<>();
        Stack<StringBuilder> stringStack = new Stack<>();
        StringBuilder current = new StringBuilder();
        int k = 0;

        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                k = k * 10 + (c - '0');
            } else if (c == '[') {
                countStack.push(k);
                stringStack.push(current);
                current = new StringBuilder();
                k = 0;
            } else if (c == ']') {
                StringBuilder decoded = stringStack.pop();
                int repeatTimes = countStack.pop();
                for (int i = 0; i < repeatTimes; i++) {
                    decoded.append(current);
                }
                current = decoded;
            } else {
                current.append(c);
            }
        }
        return current.toString();
    }
}
```

6. Algorithmic Cheat Sheet & Complexity Reference

Use this reference table in the final minutes before your technical interviews.

Algorithm / Paradigm	Best Time Complexity	Worst Time Complexity	Space Complexity	Best Used For
KMP Pattern Search	$O(N + M)$	$O(N + M)$	$O(M)$	Substring search, finding periodic prefixes.
Rabin-Karp Search	$O(N + M)$	$O(N \cdot M)$	$O(1)$	Multiple pattern matching, plagiarism detection.
Z-Algorithm	$O(N + M)$	$O(N + M)$	$O(N + M)$	Pattern matching, search prefix instances.
Trie Operations	$O(L)$	$O(L)$	$O(N \cdot L)$	Autocomplete, dictionary prefix queries.
Sliding Window	$O(N)$	$O(N)$	$O(\Sigma)$	Contiguous substring constraints (e.g., unique chars).
Edit Distance (DP)	$O(N \cdot M)$	$O(N \cdot M)$	$O(N \cdot M)$	Minimum edits, spell check, DNA alignment.